Status	Finished
Started	Tuesday, 22 September 2026, 3:24 PM
Completed	Tuesday, 22 September 2026, 4:16 PM
Duration	52 mins 24 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given a **non-empty** string s and an abbreviation abbr, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

Note:

Assume s contains only lowercase letters and abbr contains only lowercase letters and digits.

Example 1:

Input

internationalization
i12iz4n

Output

true

Explanation

Given **s** = "internationalization", **abbr** = "i12iz4n":

Return true.

Example 2:

Input

apple
a2e

Output

false

Explanation

Given **s** = "apple", **abbr** = "a2e":

Return false.

Answer: (penalty regime: 0 %)

```python
import sys

def valid_word_abbreviation():
    # Read both lines from standard input
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    s = input_data[0].strip()
    abbr = input_data[1].strip()

    i, j = 0, 0
    m, n = len(s), len(abbr)

    while i < m and j < n:
        if abbr[j].isdigit():
            # Leading zeros are not allowed in a valid abbreviation number
            if abbr[j] == '0':
                print("false")
                return

            # Parse the complete multi-digit number
            num = 0
            while j < n and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1

            # Skip the corresponding number of characters in the original string
            i += num
        else:
            # Characters must match exactly
            if s[i] != abbr[j]:
                print("false")
                return
            i += 1
            j += 1

    # The abbreviation is valid only if both pointers reached the end exactly
    if i == m and j == n:
        print("true")
    else:
        print("false")

if __name__ == "__main__":
    valid_word_abbreviation()
```

	Input	Expected	Got	
✓	internationalization i12iz4n	true	true	✓
✓	apple a2e	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

The program must accept **N** series of keystrokes as string values as the input. The character **^** represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

Boundary Condition(s):

1 <= N <= 100
1 <= Length of each string <= 100

Input Format:

The first line contains the integer N.
The next N lines contain a string on each line.

Output Format:

The first N lines contain the string after applying the undo operations.

Example Input/Output 1:

Input:

3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^

Output:

Hey google
luckycharms
-1

Answer: (penalty regime: 0 %)

```
def solve():
    # Read the number of strings
    n = int(input())

    # Process each string
    for _ in range(n):
        s = input()
        stack = []

        for char in s:
            if char == '^':
                if stack:
                    stack.pop()
            else:
                stack.append(char)

        # Check if the final string is empty
        if stack:
            print("".join(stack))
        else:
            print("-1")

```

```
23 if __name__ == "__main__":
24     solve()
25
```

	Input	Expected	Got	
✓	3 Hey ^ goooo^^glee^ lucke^y ^charr^ms ora^^nge^^^^	Hey google luckycharms -1	Hey google luckycharms -1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

Sample Input 1

a2b4c6

Sample Output 1

aabbbbccccc

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read the raw string from input
    input_str = sys.stdin.read().strip()
    if not input_str:
        return

    original_length = len(input_str)

    # 1. Compute target expanded length
    expanded_length = 0
    k = 0
    while k < original_length:
        k += 1  # Skip character
        num = 0
        while k < original_length and input_str[k].isdigit():
            num = num * 10 + int(input_str[k])
            k += 1
        expanded_length += num

    # 2. Simulate pre-allocated buffer ("given string has enough memory")
    # We allocate the exact slot size needed for in-place construction
    buffer = list(input_str) + [""] * (expanded_length - original_length)

    # 3. Two-pointer backward expansion
    i = original_length - 1
    j = expanded_length - 1

    while i >= 0:
        # Parse the multi-digit number backwards
        num = 0
        multiplier = 1
        while i >= 0 and buffer[i].isdigit():
            num += int(buffer[i]) * multiplier
            multiplier *= 10
            i -= 1

        # Get the character preceding the digit sequence
        char = buffer[i]
        i -= 1

        # Write characters backwards in-place
        for _ in range(num):
            buffer[j] = char
            j -= 1

    # Print the exact expected string without any extra debug text
    print("".join(buffer[:expanded_length]))

if __name__ == "__main__":
    solve()
```

	Input	Expected	Got	
✓	a2b4c6	aabbbbcccccc	aabbbbcccccc	✓
✓	a12b3d4	aaaaaaaaaaaabbbdddd	aaaaaaaaaaaabbbdddd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

For example:

Test	Result
`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`
`print(checkPangram('arvijayakumar'))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def checkPangram(s):
    # Check if the count of unique letters is exactly 26
    is_pangram = len(set(s)) == 26

    # Convert True/False to lowercase string "true"/"false" to match expected output
    return str(is_pangram).lower()
''
```

	Test	Expected	Got	
✓	`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`	`true`	✓

	Test	Expected	Got	
✓	`print(checkPangram('arvijayakumar'))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

For example:

Input	Result
break	break is a keyword
IF	IF is not a keyword

Answer: (penalty regime: 0 %)

```
import sys

def check_keyword():
    # Read the input string from standard input and remove leading/trailing whitespace
    word = sys.stdin.read().strip()

    if not word:
        return

    # Pre-defined set of unique keywords
    keywords = {
        "break", "case", "continue", "default", "defer", "else",
        "for", "func", "goto", "if", "map", "range",
        "return", "struct", "type", "var"
    }

    # Check if the exact string matches any element inside the set
    if word in keywords:
        print(f"{word} is a keyword")
    else:
        print(f"{word} is not a keyword")

if __name__ == "__main__":
    check_keyword()
```

	Input	Expected	Got	
✓	break	break is a keyword	break is a keyword	✓
✓	IF	IF is not a keyword	IF is not a keyword	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Note: For the purpose of this problem, we define empty string as valid palindrome.

Example 1:

```
Input:
A man, a plan, a canal: Panama

Output:
1
```

Example 2:

```
Input:
race a car

Output:
0
```

Constraints:

- `s` consists only of printable ASCII characters.

Answer: (penalty regime: 0 %)

```
import sys

def is_palindrome():
    # Read the full line from standard input
    s = sys.stdin.read().strip()

    left = 0
    right = len(s) - 1

    while left < right:
        # Move left pointer forward if the character is not alphanumeric
        while left < right and not s[left].isalnum():
            left += 1

        # Move right pointer backward if the character is not alphanumeric
        while left < right and not s[right].isalnum():
            right -= 1

        # Compare characters by ignoring case
        if s[left].lower() != s[right].lower():
            print(0)
            return

        left += 1
        right -= 1

    print(1)

if __name__ == "__main__":
    is_palindrome()
```

	Input	Expected	Got	
✓	A man, a plan, a canal: Panama	1	1	✓
✓	race a car	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]`.

You may assume all the characters consist of printable ascii characters.

Example 1:

```
Input:
hello
Output:
olleh
```

Example 2:

```
Input:
Hannah
Output:
hannaH
```

Answer: (penalty regime: 0 %)

```
import sys

def reverse_string():
    # Read the string from standard input
    input_str = sys.stdin.read().strip()
    if not input_str:
        return

    # Convert the string into a list of characters (char[] representation)
    chars = list(input_str)

    # Initialize two pointers
    left = 0
    right = len(chars) - 1

    # Swap elements from outside in
    while left < right:
        chars[left], chars[right] = chars[right], chars[left]
        left += 1
        right -= 1

    # Print the final reversed string
    print("".join(chars))

if __name__ == "__main__":
    reverse_string()

```

	Input	Expected	Got	
✓	hello	olleh	olleh	✓
✓	Hannah	hannaH	hannaH	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Given a string S which is of the format USERNAME@DOMAIN.EXTENSION, the program must print the EXTENSION, DOMAIN, USERNAME in the reverse order.

Input Format:

The first line contains S.

Output Format:

The first line contains EXTENSION.
The second line contains DOMAIN.
The third line contains USERNAME.

Boundary Condition:

1 <= Length of S <= 100

Example Input/Output 1:

Input:

abcd@gmail.com

Output:

com
gmail
abcd

For example:

Input	Result
arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read only one line to avoid picking up trailing text from other test runs
    line = sys.stdin.readline().strip()
    if not line:
        return

    # Split the string at the '@' character to isolate the username
    username, domain_part = line.split('@', 1)

    # Split the domain part at the first '.' to separate DOMAIN and EXTENSION
    domain, extension = domain_part.split('.', 1)

    # Print the results in the exact requested reverse order
    print(extension)
    print(domain)
    print(username)

if __name__ == "__main__":
    solve()

```

	Input	Expected	Got	
✓	abcd@gmail.com	com gmail abcd	com gmail abcd	✓
✓	arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar	edu.in rajalakshmi arvijayakumar	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

1 <= s.length <= 10^4

s consists of parentheses only '()[]{}'.

For example:

Test	Result
`print(ValidParenthesis("()"))`	`true`
`print(ValidParenthesis("()[]{}"))`	`true`
`print(ValidParenthesis("(]"))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def ValidParenthesis(s):
    stack = []
    # Map each closing bracket to its corresponding opening bracket
    mapping = {")": "(", "}": "{", "]": "["}

    for char in s:
        if char in mapping:
            # Pop the top element if stack is not empty, else assign a dummy value
            top_element = stack.pop() if stack else '#'

            # If the mapping doesn't match the popped element, it's invalid
            if mapping[char] != top_element:
                return "false"
        else:
            # It's an opening bracket, push it onto the stack
            stack.append(char)

    # If the stack is empty, all brackets were matched properly
    is_valid = len(stack) == 0
    return "true" if is_valid else "false"

```

	Test	Expected	Got	
✓	`print(ValidParenthesis("()"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("()[]{}"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("(]"))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

Sample Input 1

thistest123string

123

Sample Output 1

8

Answer: (penalty regime: 0 %)

```
import sys

def find_substring(s1,s2):
    return s1.find(s2)

lines = [ln.strip() for ln in sys.stdin if ln.strip()]
for idx in range(0,len(lines),2):
    if idx+1<len(lines):
        print(find_substring(lines[idx],lines[idx+1]))
```

	Input	Expected	Got	
✓	thistest123string 123	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.